

SO(3) Equivariant Neural Networks

Lecture 4

From Couplings to a Neural Network

Equivariant layers, channel mixing, and scalar gates

90-minute journal-club lecture note

Phong Dang • prepared with AI assistance

Companion notebook included in the series package

Session map and learning goals

hook	layer rules	physics	break	architecture	notebook	discussion
0			45			90 min

By the end of this lecture, you should be able to:

- list the symmetry-safe operations used in the notebook layer;
- distinguish fixed CG tensors from learned channel-mixing weights;
- trace tensor shapes through the network's main `einsum` operations;
- explain why scalar gates preserve equivariance and componentwise activations generally do not;
- give a compositional proof that the complete network is equivariant.

Key idea

An equivariant architecture is defined as much by what is absent as by what is present: there is no arbitrary weight matrix on magnetic components and no uncontrolled mixing between irrep types.

1 Opening puzzle: why not use one dense layer?

Suppose a hidden state contains two scalars, two vectors, and two quadrupoles. Flattening it produces $2 + 6 + 10 = 18$ numbers. A generic 18×18 matrix can mix all of them, but it does not know which numbers are scalar components, which form a vector, or which belong to a quadrupole.

An equivariant layer preserves this type structure. It permits only operations proven to commute with rotations.

2 Operation 1: mix channels within one irrep

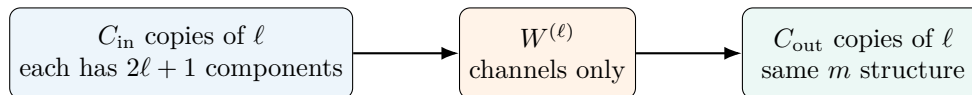
Let $x_{bm}^{(\ell)}$ denote channel b and magnetic component m . A learned matrix may act on the channel index:

$$y_{am}^{(\ell)} = \sum_b W_{ab}^{(\ell)} x_{bm}^{(\ell)}. \quad (1)$$

The same $W^{(\ell)}$ is used for every m . Therefore

$$W^{(\ell)} D^{(\ell)}(R) x = D^{(\ell)}(R) W^{(\ell)} x, \quad (2)$$

because the two matrices act on different axes.



AI lens

Channel count is analogous to width in an ordinary neural network. Increasing channels adds learned capacity without changing how a feature transforms. The $2\ell + 1$ representation dimension is fixed by symmetry, not by a hyperparameter.

3 Operation 2: couple irreps through CG tensors

For each legal path $(\ell_1, \ell_2) \rightarrow L$, first compute every channel pair:

$$z_{cdM}^{(L)} = \sum_{m_1, m_2} C_{\ell_1 m_1, \ell_2 m_2}^{LM} x_{cm_1}^{(\ell_1)} x_{dm_2}^{(\ell_2)}. \quad (3)$$

Then mix the path channels with learned weights:

$$y_{aM}^{(L)} = \sum_{c,d} W_{a,cd}^{(\ell_1, \ell_2, L)} z_{cdM}^{(L)}. \quad (4)$$

The CG tensor determines the angular coupling; W selects task-dependent combinations of channels.

3.1 Reading the main einsum

```
products = np.einsum(
    'Mij,bci,bdj->bcdM', C, features[l1], features[l2]
)
```

Index	Meaning
b	batch/sample
c, d	channel indices of the two inputs
i, j	magnetic indices m_1, m_2 contracted by the CG tensor
M	magnetic index of the output irrep L

The output initially has shape

$$(B, C_{\ell_1}, C_{\ell_2}, 2L + 1). \quad (5)$$

The two channel axes are flattened and mixed with learned weights. The M axis remains protected.

4 Operation 3: symmetry-safe nonlinearities

Ordinary neural networks rely on nonlinear activations, but a componentwise nonlinearity generally fails for non-scalar irreps:

$$\sigma(D^{(\ell)} x) \neq D^{(\ell)} \sigma(x), \quad \ell > 0. \quad (6)$$

The reason is simple: $D^{(\ell)}$ mixes components, whereas a componentwise activation transforms each number separately.

The notebook uses scalar gates. From invariant scalar features $s = x^{(0)}$, compute

$$g = \tanh(W_g s + b_g), \quad (7)$$

then rescale a complete multiplet:

$$y_{cm}^{(\ell)} = g_c x_{cm}^{(\ell)}. \quad (8)$$

Because g_c is invariant and independent of m , Equation (8) is equivariant.

It is worth knowing that this is not a clever trick among many. It is the only thing available.

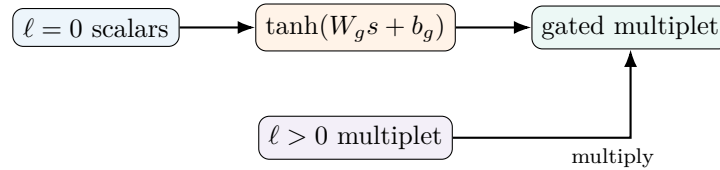
Math lens

Suppose $f : \mathbb{R}^3 \rightarrow \mathbb{R}^3$ satisfies $f(Rx) = Rf(x)$ for every rotation. Then necessarily

$$f(x) = g(\|x\|) x \quad (9)$$

for some scalar function g . The argument is short: $\|x\|$ is the only invariant a single vector has, and by Schur's lemma the only *linear* equivariant maps on an irrep are multiples of the identity. So “multiply the whole multiplet by a function of invariants” is not one design option; it is the general form of an equivariant map on one irrep. With several channels it becomes $f_c = \sum_d g_{cd}(\{x_a \cdot x_b\}) x_d$: arbitrary functions of the pairwise invariants, multiplying the vectors.

This reframes Question 4.4. The interesting question is not why a scalar gate happens to work, but why nothing else can.



Physics lens

The gate resembles an orientation-independent material response coefficient. It may depend nonlinearly on invariant state variables, but it scales an entire directional object without selecting a preferred axis.

4.1 What the gate actually adds

Lecture 3 showed that the CG coupling is bilinear, so a network of coupling layers is already nonlinear with no activation at all — unlike an MLP, which collapses. It is fair to ask what the gate is for.

The answer is that the tensor product supplies *polynomial* nonlinearity and nothing else. Two coupling layers reach degree four and stop. A gate multiplies by $\tanh$ of an invariant, which is not a polynomial of any degree, so it contributes saturation, thresholds and decay — the behaviour you need when a response should level off rather than grow without bound. The two mechanisms are complementary: couplings build geometric structure, gates shape the invariant dependence.

The notebook measures exactly this. A homogeneous polynomial of degree d satisfies $f(\lambda x) = \lambda^d f(x)$, so the slope of $\log\|f\|$ against $\log \lambda$ reads as a degree. With the coupling paths switched off the slope is a flat 1 at every scale — the MLP collapse, made visible. With couplings and no activation it climbs from 1 to 4 *monotonically* and stops exactly there. With a gate it does neither: the curve leaves $[1, 4]$ and stops being monotone.

Read that last comparison carefully, because only half of it is robust. The size of the excursion is an artifact of the random weights — across seeds the same architecture gives a maximum slope anywhere

from 4.1 to 6.6 — so no single overshoot number is a result. What survives every seed is the *shape*: the un-gated network is monotone to machine precision, and no gated one is. That contrast is between two networks differing in exactly one component, which is all the argument needs.

AI lens

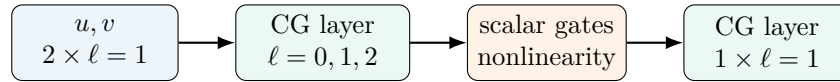
The gate may read *any* invariant, not just the $\ell = 0$ block. Every $\ell > 0$ channel also supplies $\|x\|^2 = \sum_m |x_m|^2$, invariant because $D^{(\ell)}$ is unitary and free to form — no CG table required. The notebook builds both gates, $\tanh(W \cdot s + b)$ and $\tanh(W \cdot [s, \|x\|^2] + b)$, and compares them: both are exactly equivariant, and they differ mainly in how fast the gate argument grows with input scale, since $\|x\|^2$ is two degrees higher than s . Norm-only gating is the classical “norm nonlinearity”, and it is Equation (9) written differently — but note it is *less* informative than the scalar gate, because a channel can then only be scaled by its own magnitude and not by an invariant computed elsewhere in the layer.

A genuinely different family evaluates the feature on a grid of directions on the sphere, applies an ordinary pointwise nonlinearity there, and projects back onto irreps; this is what spherical CNNs use, and it buys expressivity at the cost of a transform in each layer.

5 The notebook architecture

The input and output multiplicities are

$$M_{\text{in}} = \{1 : 2\}, \quad M_{\text{hidden}} = \{0 : 2, 1 : 2, 2 : 2\}, \quad M_{\text{out}} = \{1 : 1\}. \quad (10)$$



The first layer creates scalar, vector, and quadrupole information from two input vectors. Scalar gates provide nonlinearity. The final layer returns one vector.

5.1 Fixed versus learned

Object	Status	Role
Irrep dimensions $2\ell + 1$	fixed	component count
Allowed $(\ell_1, \ell_2) \rightarrow L$ paths	fixed	triangle rule
CG tensors	fixed	angular component coupling
Channel counts C_ℓ	chosen	network width
Channel-mixing matrices	learned	task-dependent combinations
Scalar gate parameters	learned	nonlinear invariant modulation

6 Why the whole network is equivariant

The proof is compositional:

1. Same- ℓ channel mixing commutes with $D^{(\ell)}$.

2. CG tensor products satisfy the intertwining identity.
3. Scalar gates multiply complete multiplets by invariants.
4. Sums and compositions of equivariant maps remain equivariant.

Therefore

$$f_{\theta}(Ru, Rv) = Rf_{\theta}(u, v) \quad (11)$$

for every parameter vector θ .

Key idea

Optimization cannot break this symmetry because it changes only the learned channel weights. The protected magnetic axes and fixed intertwiners define the entire space of accessible functions.

7 Failure modes as controlled experiments

Each modification below breaks a specific rule:

Modification	Why it fails
Random tensor replaces CG tensor	no intertwining identity
Dense matrix mixes all flattened features	mixes irrep types and m components
Constant vector bias	selects a preferred direction
Componentwise tanh on a vector	does not commute with component mixing

An equivariance residual measures symmetry, not prediction accuracy. A network can be exactly equivariant and still predict the wrong physical function.

Pause and discuss

What does architectural equivariance guarantee? What important properties does it not guarantee? Consider accuracy, conservation laws, locality, permutation symmetry, and data quality.

8 Notebook laboratory

Open `lecture_04_equivariant_layers.ipynb`. You will:

1. enumerate legal CG paths from multiplicity dictionaries;
2. trace all tensor shapes through one layer;
3. assemble the two-layer network;
4. verify equivariance for random weights;
5. break each of the four rules above in turn, and read the residuals as one table;
6. measure polynomial degree from a log-log slope, and see where the nonlinearity comes from.

Every row of that table uses the same untrained weights, so the comparison is about structure alone. The valid architecture sits at roundoff; each violation lands between 10^{-2} and 10^0 .

9 Questions and discussion

Question 4.1. Why may a learned matrix mix channels of the same ℓ without breaking equivariance? Why may it not arbitrarily mix magnetic components?

Question 4.2. For input multiplicities $\{1 : 2\}$ and output multiplicities $\{0 : 2, 1 : 2, 2 : 2\}$, what is the shape of the tensor-product weight matrix for each $1 \otimes 1 \rightarrow L$ path?

Question 4.3. Explain why a scalar bias is allowed but a fixed vector bias generally breaks rotational equivariance.

Question 4.4. Why is a scalar gate nonlinear yet equivariant?

Question 4.5. A network has an equivariance error of 10^{-15} but a large test MSE. Is this a contradiction?

Question 4.6. Compare increasing channel width with increasing $\ell_{\max}$. What different kinds of capacity do they add?

Question 4.7. Prove Equation (9): if $f : \mathbb{R}^3 \rightarrow \mathbb{R}^3$ commutes with every rotation, then $f(x) = g(\|x\|)x$. Hint: fix x and consider the rotations that leave it unchanged. What must $f(x)$ be parallel to?

Checkpoint

You are ready for Lecture 5 if you can identify the protected m axis, the learnable channel axes, and the fixed CG contraction in the layer code.